

객체프로그래밍 과제 보고서

문제 1번

- 문제설명

5개의 정수를 입력 받은 후, 최댓값,최솟값,평균값을 출력하는 문제이다.

- 알고리즘

Int 형 변수 a에 다섯개의 정수를 입력 받는다. 숫자를 받으면서 크기를 비교해 min과 max를 출력할것이다. Min과 max 를 먼저 1번 정수값으로 설정해 놓은 후, 2번째 들어오는 값과 비교한다. 먼저 max 값으로 예를 들어보자. 초기 설정해놓은 max값 보다 새로 들어온 a 의 값이 크다면 max에 새로들어온 a값을 저장해 준다. 이런 방식으로 3번째숫자, 4번째 숫자, 5번째 숫자와 max 값을 비교한다. 그리고 새로들어온 숫자가 더 크다면 그 값을 max 에 저장한다. 이것을 for문과 if 문으로 구현하였다.

for (int i=1; i<5; i++) {cin>>a;} 를 통해 a를 정수를 4번 받을 것이다. 왜 5개가 아니라 4개인것인가? 그 이유는 for 문 전에 위에서 먼저 첫번째 a를 입력 받았고, 그 a 값을 max,min,avg의 초기값으로 선언했기 때문이다. 따라서 4개의 숫자를 더 받고, 그것을 각각max,min과 비교하여 최댓값과 최소값을 찾아주는것이다.

다음은 avg 를 구할 차례이다. Avg를 구할 때 고민한 것은 값을 어떻게 받고 sum을 구할것인지와 sum/5를 할 때 소수점을 처리하는 과정에서 반올림을 어떻게 구현할지 였다.

Avg를 하려면 먼저 sum을 구해야 하는데 처음 sum을 구하는 방식으로 생각이 든 것은 배열로 받아 다 저장하고 다시 각각의 자리수를 꺼내서 더하는 방식이었다. 그런데 이 방식보다 숫자를 받으면서 더한다면 훨씬 간단하겠다는 발견을 했다. 따라서 위에서 말한 for 문을 이용해 값을 받을 때 avg+=a 를 해주어 a값을 받을때마다 avg에 더해 저장해 주었다. 이때 for문에서 a 가 4번 받아지는데 그럼 다 합한게 아니지 않나라는 의문이 들 수 있지만 avg 의 초기값도 첫번째 숫자인 a 로 저장해 주었기 때문에 상관없다.

그리고 sum이라는 변수를 선언해주고 avg=sum/5 라고 해줄수도 있지만 평균값avg란 값을 다 더한후-> 개수만큼 나눈다 라는 연산과정이고 변수를 두개 쓰지 않아도 된다고 생각하여 sum 을 쓰지 않았다.

그리고 마지막으로 생각해줘야 할 부분은 avg의 반올림 처리이다.

이부분을 구현해주지 않고 출력한다면, 소수점은 반올림 되지 않고 그냥 값이 버려진다.

그래서 먼저 평균값이 소수점이 생길때와 안생길때를 나누고, 소수점이 생길때는 그 자리수값이 5보다 크거나같은지 작은지를 나누어 작을땐 버리고 클때는 반올림 해주는 알고리즘을 짰다.

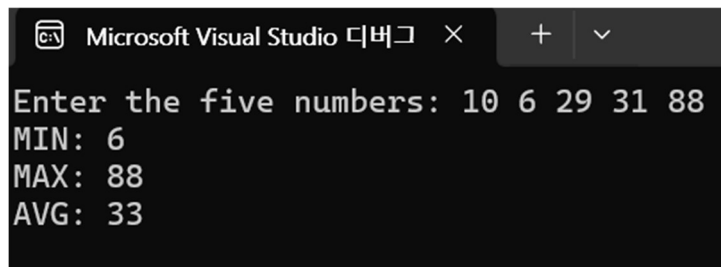
소수점이 생길 때 : $Avg \% 5 > 0$ 이라면, 즉 sum을 5로 나눈값의 나머지가 0보다 클 때이다. 이는 나머지가 생겼다는 것이고, 나머지가 생겼다는 것은 딱나누어 떨어지지 않는다는 뜻이다. 이경우는 또 반올림을 해줄지 말지로 나누어 주어야한다.

이를 if 문으로 구현하였고 if문의 조건은 다음과 같다. $(Avg \% 5) / 5.0 > 0.5$ 일 때 즉 나머지를 5로나누었을 때 0.5 보다 크다면 이는 5이상이상으로 반올림 대상이다. ****여기 좀더 자세히 써라**** 따라서 5로 나눈값에 1을 더해준다. 왜냐면 소수점은 그냥 버려졌는데 이 소수점이 0.5보다 크다는 뜻이기 때문이다. 그리고 ' $(Avg \% 5) / 5.0 > 0.5$ '이렇게 연산해줄 수 있는 이유는 avg%5가 항상 정수이기 때문이다. 그래서 캐스팅을 쓰지 않고도 계산해 줄수있다. 만약 나누는값이 avg%5 가 아니라 avg 였다면 위와 같이 계산하면 오류가 났을것이다.

소수점이 생기지 않을 때: 소수점이 생기지 않을때는 딱 나누어 떨어졌다는 뜻으로 반올림을 해줄 필요가 없다. 따라서 else 문을 이용해 $avg = avg / 5$ 라고 출력한다. 소수점이 0.5이상인 경우가 위와 같고 0.5보다 작은 경우는 else 문을 이용하여 처리해준다.

그리고 마지막으로 평균값이 음수가 나올때를 고려해 if-else문으로 나누어 처리하였다. 음수가 나올때는 -1을 곱해서 소수점 판단을 해주고 반올림을 하면 절댓값이 커지고 -를 붙인 값은 작아지는 것이니 이를 고려하여 반올림 해주었다.

- 결과 및 고찰 분석



```
Microsoft Visual Studio 디버그 × + -
Enter the five numbers: 10 6 29 31 88
MIN: 6
MAX: 88
AVG: 33
```

1) 정렬을 쓰는 것 이외의 방식 발견

나는 1번 문제에 5개의 정수를 받는다는걸 보자마자 정렬만 생각이 났다. 각 자릿수에 값을 저장하고 꺼내서 비교하는 방식만 생각이 났다. 그런데 값을 비교할 때 하나 비교하고 아니면 버린다. 이 부분에서 숫자를 계속 저장해놓을 필요가 있나? 라는 생각이 들었다. 값을 받으면서 max와 min을 비교하고, 값을 받으면서 sum을 구하면 더 간단하게 구현될 것 같다는 생각이 들었다. 물론 정렬로 받을때의 장점은 ???

있겠지만 처음 떠올린 방식외에 다른 방식을 찾아 문제를 풀었다는 점에서 새로운 문제 풀이 방식을 배웠다.

2) 소수점과 캐스팅

처음엔 평균값의 소수점을 처리하는 방식으로 (double)sum 같은 방법이 떠올랐는데 나머지를 이용한다면 캐스팅을 쓰지 않아도 구현이 가능할 것 같았다. Avg%5 , 모듈러 연산은 항상 정수로 0,1,2,3,4 중에 하나가 나온다. 이를 이용하여 반올림을 할지 말지를 if 문을 사용해 나누어 주었다. 만약에 (Avg%5)/5.0>0.5) 이 식에 avg%5 가 아니라 avg가 들어갔다면 오류가 났을 것이다.

이처럼 당연한 것을 다시한번 생각해보고 혹시 다른 방법으로 구현할 수 있지 않을까 하고 여러 방면으로 생각하는 방식이 나의 역량을 키워준다는 사실을 깨달았다. 사실 큰 차이가 없고 별거 아닐 수 있지만 다른 해법이 있나 골몰했다는 점 자체에서 의미 있다고 생각한다.

#문제 2번

- 문제 설명

숫자들을 랜덤함수를 이용해 생성하고, avg와 sum을 구하고 숫자들을 오른쪽 정렬하여 출력하는 문제이다. (여러 제약조건들을 따르면서)

- 알고리즘

난수생성 - 먼저 rand를 사용해 난수를 생성한다. 필요한 핵심변수는 avg, data이다. 숫자 출력은 2차원 배열 data[i][j]를 이용할 것이다. 이때 난수는 1~99까지 이다. 0은 제외해야 한다. 이를 위해 난수를 생성할 때 rand()%99+1 라고 구현하였다. 만약 rand()%100을 했다면 0~99까지의 난수가 생성 됐을 것이다. 그렇게 생성한 난수들을 2차원 배열에 규칙을 지키며 출력시킨다.

자릿수에 따른 공백 삽입 - 첫번째 열에 한자릿수 숫자가 온다면 한칸 공백을 띄워준다. 이것을 위해 첫번째 행일때와 아닐때를 if 문으로 나누어 작성하였다. 그렇게 한 행의 출력이 끝나면 행 총합을 구해주고 행 총합에 따라 공백과 기호 | 를 출력한다.

반올림 - avg를 다룰 때는 반올림을 고려해 주어야 한다. 위의1번 문제에서 반올림을 처리해준대로 (avg%10) / 10.0 >0.5 를 조건으로 하는 if 문을 만들어서 반올림을 해주는 경우와 아닌경우를 나누어 연산한다.

- 결과 및 고찰 분석

83	78	65	49	26	86	69	71	24	92		643		64
30	81	22	28	39	18	5	48	28	25		324		32
72	18	73	32	27	24	70	10	26	8		360		36
3	51	7	89	86	34	8	70	96	28		472		47
6	25	54	3	8	11	5	6	83	44		245		24

결과 고찰: 결과 화면을 보면 알 수 있듯이 이 문제는 난수들을 출력하고 연산하여 오른 쪽 정렬해 출력하는 것이 포인트이다. 이를 수행하기 위해서 생각해야할 점들은 난수 생성, 자릿수에 대한 공백 삽입, 반올림 연산이다.

처음 문제를 받았을 땐 난감하고 너무 어렵게 느껴졌다. 그러나 문제를 풀기 위해서 생각해야할 포인트들을 찾아내고 이를 어떻게 구현할지 생각하니 문제가 풀리기 시작했다. 출력값을 보면 자릿수에 따른 공백을 어떻게 할것인지 생각하고, 1~99까지의 랜덤숫자들을 어떻게 생성할것인지 생각하고, 반올림 처리에 대해서만 고민하면 구현이 가능했다. 물론 이와 같은 과정에서 시행착오도 있었다. 처음에 난수 발생할 시킬 때 0이 포함 됐다던지, 숫자가 한칸씩 밀린다던지 하는 문제들도 생겼었다. 하지만 틀리고 코드를 고치는 과정에서 문제를 인식하는 방법에 대한 능력이 늘었다는걸 깨달았다.

#문제 3번

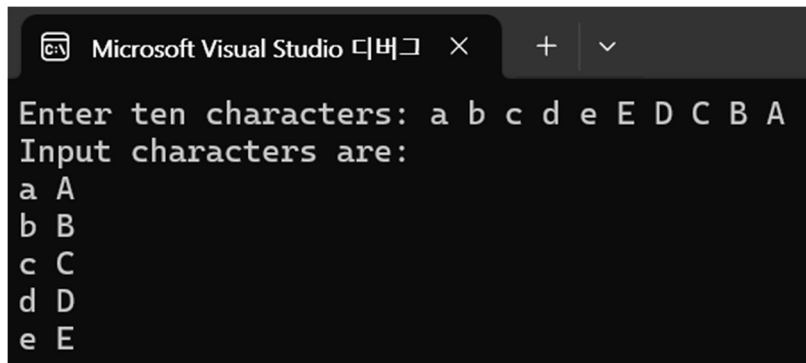
- 문제 설명

이 문제는 10개의 문자를 입력받고, 1번과 10번을 짝지어 한줄에 출력하고, 2번과 9번, 3번과 8번, 이런식으로 각각 짝지어 5행2열의 모양으로 출력하는 문제이다.

- 알고리즘

먼저 10칸짜리 문자열로 문자들을 입력 받는다. 첫 번째 (0번지) 문자를 head로 마지막(9번지) 문자를 tail로 지정하고 head에는 0, tail에는 9를 초기값으로 입력하고 , for문을 활용해 짝지어서 꺼낸다. 0번 9번/ 1번 8번 이런식으로 출력하기 위해 for 문을 돌때마다 head는 초기값0에서 1씩 더해주고 tail은 9에서 1씩 빼준다. 그리고 for문의 조건은 (int j=0; j<5; j++) 으로 5번 돌게 한다. 그래야 2개씩 5번 짝지어서 나온다. 종료시점은 (n>9-n이 될때이다.)

- 결과 및 고찰 분석



```
Microsoft Visual Studio 디버그 × + ▾  
Enter ten characters: a b c d e E D C B A  
Input characters are:  
a A  
b B  
c C  
d D  
e E
```

이 문제를 풀 때 변수를 어떻게 선언할지 고민했었다. Head와 tail 두개로 변수를 선언하고 하나는 1씩 증가, 하나는 1씩 감소 하는 방식도 있지만, 변수를 애초에 head 하나만 정하고 나머지는 $n - \text{head}$ 이런 방식도 있으니 둘 중에 어떤 방식이 좋을지 고민하였다. 이 방식은 변수를 하나만 설정해 줘도 된다는 장점이 있지만, 변수를 두개 설정했을 때 가독성이 좋다고 생각해 전자의 방식으로 코드를 작성하였다. 어떤 코드가 좋은 코드일지 깊이 고민해보게 되는 부분이었다.

그리고 처음에는 for문으로 두개씩 짝지어서 꺼낼 때 for 문의 조건을 ($\text{int } j=0; j<9; j++$)로 설정하는 실수를 해 출력이 이상해지는 결과가 나왔다. 어렵지 않다고 생각한 문제도 정말 생각지도 못한 실수를 할 수 있다는걸 다시금 깨달았다. 기계적으로 코딩하는 것이 아니라 조건 하나하나 고려해서 코딩할 수 있도록 노력해야겠다는 생각을 했다.

#4번 문제

- 문제 설명 :

양의 정수를 입력받아 거꾸로 출력하는 문제이다. 예를 들면 1234는 4321로 출력한다. 문자로 출력하는 것이 아니라 숫자로 출력해야한다. 따라서 2200은 0022가 아니라 22로 출력되어야 한다.

- 알고리즘

먼저 숫자 8263를 입력받아 3628로 출력하려면 각각의 자리수를 숫자 하나씩 뜯고 거꾸로 뒤야한다. 그럼 숫자를 한번에 찢고 따로따로 저장한 후 자릿수에 맞춰 하나씩 불러오는게 좋을까? 그것보다는 일의 자리 3이 천의 자리 4로 가는 과정을 생각해보고 이를 구현하는 것이 좋다고 판단했다.

따라서 먼저 변수 num , temp, result를 선언해준다. 각각 몫, 나머지, 결과값이다. 우리는 이 결과값에 숫자를 뒤집어 저장하는 것이다.

가지치기 그림 넣을까?

먼저 8263에서 일의 자릿수 3을 떼어내기 기해 8363 나누기 10 을 해준다. 그러면 몫은 826 나머지는 3이 나온다. 따라서 $num/10=몫$, $num\%10=나머지$ 를 구할 수 있는 것이다. 그리고 다시 몫 826을 10으로 나눈다면 몫 82와 나머지 6을 구할 수 있다. 또 82를 10으로 나눈다면 몫 8과 나머지 2를 구할 수 있다. 한번 더 몫 8을 10으로 나눈다면 이번엔 몫 0과 나머지 8이 생긴다. 나머지들을 살펴보자. 순서대로 3-6-2-8 이 되는걸 알 수 있다. 하나씩 떼어 내는건 됐지만 이제 애네한테 자릿수를 어떻게 입힐것인가? 이걸 3을 떼어냈을 시점부터 생각해보자. 3을 먼저 받아서 1의자리에 두면 한번씩 나누기 연산을 하고 다른 나머지값을 받을때마다 왼쪽으로 한칸 씩 밀려나는걸로 볼 수가 있다.

	3
	$3*10 + 6$
	$3*100 + 6*10 + 2$
	$3*1000 + 6*100 + 2*10 + 8$

왼쪽과 같이 3을 보면 한칸씩 왼쪽으로 이동하는걸 알 수 가 있다. 따라서 값을 다 받고 10^n 을 곱해서 자릿수를 부여해주지말고 이렇게 num을 10으로 나눈 나머지는 그 전 식에 10을 곱한 후 더해주고 , 몫은 다시 num에 저장해 또 10으로 나누어 주는 연산을 계속해서 하는것이다. 언제까지? 몫이 0이 될 때 까지. 그걸 식으로 표현하면 다음과 같다.

(나머지) $temp = num \% 10$, (몫) $num = num / 10$ ㅎㅎ

$result = result * 10 + temp$

위 식을 통해 결과값 result를 구할 수 있다. 몫이 0이 될 때가 종료조건인 반복문에 위의 식을 넣으면 원하는 값을 구할 수 있다.

그러면 숫자로서 순서를 뒤집을 수 있다. 만약 숫자로서 뒤집지 않고 그냥 문자로 순서만 바꾸어 출력했다면 2630→ 362나 2300→32와 같은 연산은 불가능 했을 것이다. 문제의 포인트는 숫자로서 출력하는것이기 때문이다.

그리고 왜이렇게 한칸씩 왼쪽으로 밀려나게끔 했나면, 입력이 몇자리 짜리 숫자인지 정해지지않았으니 일의자리를 한번에 천의자리로 보내는게 불가능하다. 천의자리 까지 있는 숫자일지 만일지 십만일지 어떻게 알고 그때마다 첫번째 자리로 한번에 보내주겠는가? 따라서 한칸씩 밀려나는 방식으로 식을 구성했다. 반복문을 구성해보면 다음과 같다.

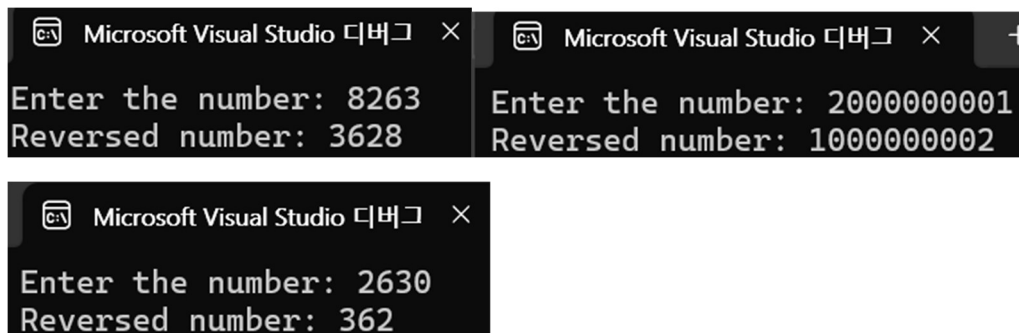
```
while(num != 0)

{ temp = num % 10;

num = num / 10;

result = result * 10 +temp; }
```

- 결과 및 고찰 분석



이 문제를 풀 때 내가 막혔던 부분은 크게 두개가 있다. 첫번째는 각 자릿수의 숫자들을 어떻게 찢을것인가 였고, 두 번째로는 찢은 각각의 숫자들을 어떻게 다시 거꾸로 배열할 것인지 였다. 숫자를 찢는 방법은 여러가지가 있지만 식을 하나만 사용하고, 어떤 숫자가 들어와도 적용되게끔 하는 방법을 찾는데 고민하였다. 코딩 과제를 하는데 수학적 논리적 사고가 정말 중요하다는 것을 다시금 느꼈다. 이렇게 코딩으로 구현하기전 문제를 분석하고 어떻게 풀것인지 생각하는 과정에서 숫자놀이를 하는데 재미를 느꼈고, 기계적이 코딩하는 것이 아니라 수학적으로 진리를 찾는게 먼저라는 것을 깨달았다. 특히 두 번째 고민인 찢은 숫자들의 자리를 재배열 하는 방식을 고민하는 것이 오래걸렸다. 처음에는 문자열로 받아 거꾸로 꺼내주는 방식을 생각했다. 그리고 예제의 3620 같은 경우는 마지막 자리가 0일 때로 if 문을 만들어 예외처리 해주면 되겠다고 생각했는데 그렇게 되면 2300 같은건 구현이 안되겠구나 생각이 들어 뒤엎었다. 그렇게 고민하다 입력값을 10으로 나누고 몫과 나머지로 저장하고 연산하면서 한칸씩 밀어주면 되겠다는 생각이 떠올랐을때는 정말 기뻐다. 문제를 풀 때 나름의 규칙을 찾는게 먼저라는 생각이 들었다.

#5번 문제

- 문제 설명: 0 ~ 32767 사이의 숫자들을 입력???? 받은 후 규칙에 따라 출력한다. 숫자 사이에는 2개의 공백을 삽입하고, 행이 아래로 내려갈수록 높은 자리 숫자들은 왼쪽을 한 칸씩 밀려 사라지는 형식으로 출력한다. 첫번째 행은 5개의 숫자 두번째 행은 4개의 숫자 이런식으로 출력하여 마지막 다섯번째 행에는 일의자리 숫자 한 개가 출력되도록한다.

만약에 입력받은 값이 1234와 같이 5섯자리 미만의 숫자라면 앞의 빈 자리에는 0을 채워 출력을 시작한다.

- 알고리즘

이 문제는 4번과 유사한 점이 있다. 다른 점으로는 형식 틀이 정해져있다는 것이다. 그렇게 되는 이유는 어떤 수(범위내의)를 받던 5자리 숫자로 간주하고 출력을 시작하기 때문이다. 따라서 전체???? 행에 대한 for 문 조건을 `for (int i = 5; i > 0; i--)`로 설정했다. 그리고 한 행에대한 for문 조건을 `for (int j = 1; j <= i; j++)`로 두어서, 첫번째 행일 때 `i=5`가 되고 `j`가 5번 돌게 되어 5개의 숫자가 출력된다. 그렇게 각 행에 맞는 개수의 숫자가 출력된다. 그럼 이제 각 자리수를 어떻게 출력할 것인가?

1234로 예를 들어보자. 만약 1234를 입력받았을 때, 0은 1234를 10000으로 나눴을 때 몫이고 1234는 나머지가 된다. 0은 해결되었다. 그다음 1은 1234를 1000으로 나눴을 때 몫이다. 나머지는 234이다. 이를 다시 100으로 나눴을 때 몫은 2, 나머지는 34이다. 34를 10으로 나눴을 때 몫은 3 나머지는 4이다. 몫 3을 다시 1로 나누면 몫은 0으로 마무리가 된다.

규칙이 보이지 않은가? 입력받은 숫자를 1행은 10^4 으로 나누고 몫을 1번에 출력하고 나머지는 다시 나눌값으로 저장해준다. 이렇게

- 결과 및 고찰 분석 : 분석은 위에 알고리즘 카테고리에 겹쳐썼다.

```
Enter the number: 1234
0  1  2  3  4
1  2  3  4
2  3  4
3  4
4
```

#6번 문제

- 문제 해설

표준 라이브러리 함수를 사용하지 않고 부동 소수점 숫자를 읽고 최대값, 바닥 및 반올림 연산을 하여 출력하도록 한다. 이 프로그램은 소수점 이하 두 자리 까지 계산한다. 소수점 셋째 자리가 5보다 작으면 절대값이 반올림 됩니다. 소수점 셋째 자리가 5 또는 5보다 크면 절대값이 반올림 된다. 이 점을 유의하여 코딩하자.

- 알고리즘

숫자를 입력받아 올림 내림 반올림을 연산해주도록 만들면 된다. 입력 받은 숫자는 자릿

수 최대가 정해져 있기 때문에 1000을 곱해서 10으로 나누면 일의 자리숫자 즉 반올림 올림 내림 여부를 판단해주는 정수가 하나 떨어진다. 이 숫자의 크기를 판단 해준 올림 내림을 판단 한다. 음수는 절대값을 붙여서 연산하기 때문에 올림 내림 반올림 연산할 때 주의해줘야한다. 절대값의 크기가 커지면 값은 작아지는 것이기 때문이다. 음수를 -1을 곱해서 양수바꾸고 올림, 내림을 해보면 원래 양수일 때랑 반대가 된다. 예를 들어 -2.545(소수점 둘째자리까지)이면 내림을 하면 -2.55이다. 반면 2.545를 올림 하면 2.55가 된다. 따라서 올림 내림 연산을 할 때 음수와 양수를 따로 나누고 연산하지 않고 음수는 양수의 반대로 따라가 주면 된다.

- 결과 및 고찰 분석

```
Enter the floating-point number: 12.234
Celing 12.24
Floor 12.23
Rounding 12.23
```

```
Enter the floating-point number: 1.035
Celing 1.04
Floor 1.03
Rounding 1.04
```

```
Enter the floating-point number: -1.035
Celing -1.03
Floor -1.04
Rounding -1.04
```

올림과 내림 반올림에 대해 깊게 생각해 볼 수 있었다. 특히 음수를 처리할 때 값이 나오지 않아 음수 처리를 또 나누어서 코딩해야겠다고 생각했는데 양수일 때 와 반대로 따라가 주면 된다는 것을 깨달을수 있어서 충격이었다. 그리고 여러 장치들을 곱하고 추가 하면서 재미를 느꼈다. 분석은 위에 알고리즘 카테고리에 겹쳐썼다.

#7번 문제

- 문제해설: 호출될 때마다 $base^{exponent}$ 를 반환하는 재귀함수를 작성하는 문제이다. 예를들어 $power(3,4)$ 는 $3 \times 3 \times 3 \times 3$ 이다. 지수에 대한 정의와 $base^0=1$ 임 을 이용하여 종료조건을 설정하여라.

- 알고리즘

재귀함수를 이용해 연산하고 그 값을 출력하는 프로그램이다. 재귀함수란, 종료조건 전까

찌 자기 자신을 계속해서 호출하여 연산하는 함수이다. 예를들어 $p(3,4)$ 는 $3 \times 3 \times 3 \times 3$ 와 같다. 이것을 재귀함수 알고리즘으로 이해를 돕기 위해 써보자면

$P(3,4) \rightarrow 3 \times p(3,3) \rightarrow 3 \times 3 \times p(3,2) \rightarrow 3 \times 3 \times 3 \times p(3,1) \rightarrow 3 \times 3 \times 3 \times 3 \times p(3,0)$ 로 나타낼 수 있다.

└ b번 ┐

이때 $p(3,0)=3^0=1$ 임으로 종료조건이고 재귀를 종료한다. 이것을 if 문으로 구현하였다. 지수가 0이 아닐때는 계속 지수에 -1를 하며 호출은 반복하고, 지수가 0이 되면 1을 반환하면서 호출을 종료한다.

- 결과 및 고찰 분석

```
Enter the exponent: 3
Enter the power: 4
power(3,4): 81
```

이 문제를 풀며 재귀함수가 어떻게 동작하는지 알게 되었다. 이것을 이용하면 반복문을 대체하여 문제를 풀 수 도 있겠다는 생각이 들었다. 분석은 위에 알고리즘 카테고리에 겹쳐썼다.

#8번

- 문제 해설

두 정수 x, y 를 입력 받아 두 수의 최대공약수를 구하는 문제이다. 최대 공약수를 구할 때는 재귀적으로 정의되는 재귀함수 gcd 를 이용하도록 한다. Gcd에 대한 조건은 다음과 같이 제시해주었다. y 가 0이면 $\text{gcd}(x, y)$ 는 x 이다. 그렇지 않으면 $\text{gcd}(x, y)$ 는 $\text{gcd}(y, x \% y)$ 이다.

- 알고리즘

두 수의 최대 공약수를 구하기 위한 연산을 재귀함수로 구성해 주었다. 두 수의 최대공약수는 유클리드 호제법 으로서 이것을 간단하게 재귀함수로 구성할 수 있다.

$\text{gcd}(x, y)$ 는 $\text{gcd}(y, x \% y)$ 가 함수 양식이고 종료조건은 $y=0$ 이 될 때 이다. 이것을 if 문으로 구현해 준다. $x \% y$ 모듈러 연산을 계속해서 해주는데 종료될 경우는 return first를 하도록 종료조건을 설정해 주었다.

- 결과 및 고찰 분석

```
Enter the 1st number: 12
Enter the 2nd number: 18
gcd(12,18): 6
```

최대공약수를 구하는 방식을 재귀함수로 구현하면서 재귀함수에 대한 이해를 보다 깊게 할 수 있어서 좋았다. 그리고 고등학교 때 유클리드 호제법에 대해 배웠는데 그것이 이렇게 재귀함수와 연결지어 구현할 수 있다는 사실이 흥미로웠다. 이러한 재귀함수의 특성을 더 공부한다면 다른 문제를 해결할 때 재귀함수의 방식이 유리하다면 이를 적용하여 풀 수 있을 것 같다. 예를 들면 큰 함수를 계산해야 할 때 필요한 작은 함수들이 뭔지 알아내고 거꾸로 그 함수들을 이용하여 연산을 하는 방식으로 말이다. 분석은 위에 알고리즘 카테고리에 겹쳐썼다.

#9번 문제

- 문제 해설

정수 x, y 를 입력받아 두 수의 최소 공배수를 구해야 한다. 최소 공배수를 구할 때는 8번의 gcd 함수를 이용하여 x 와 y 의 최소 공배수를 반환하는 함수 $\text{lcm}(x, y)$ 를 구한다.

- 알고리즘

먼저 최대공약수 $\times$ 최소공배수는 두수의 곱이라는 것을 이용하여 문제를 풀 것이다. 왜 이렇게 되냐면, 두 수를 곱한 값에서 교집합을 빼준게 최소공배수이기 때문이다. 여기서 교집합은 최대공약수이다. 따라서 $x * y = \text{lcm}(x, y) * \text{gcd}(x, y)$ 임을 이용해 코드를 구현했다. 먼저 재귀함수 gcd에 대한 조건과 선언을 해주고 함수 lcm을 작성한다. 이는 재귀로 적어도 되지만 아까 gcd 처럼 if와 else로 나누어 주지 않아도 된다. 항상 최소공배수는 두수의 곱 나누기 최대공약수 이기 때문이다. 그리고 형식에 맞춰 출력한다.

- 결과 및 고찰 분석

```
Enter the 1st number: 12
Enter the 2nd number: 18
lcm(12,18): 36
```

최대공약수와 최소공배수의 수학적 관계를 알고 재귀함수를 사용하는 방식을 익혔다. 재귀함수가 익숙하지 않아 사용이 낯설었지만 이를 잘 사용하면 반복문 대신 사용할수있겠다는 생각이 들어 재미있었다. 분석은 위에 알고리즘 카테고리에 겹쳐썼다.

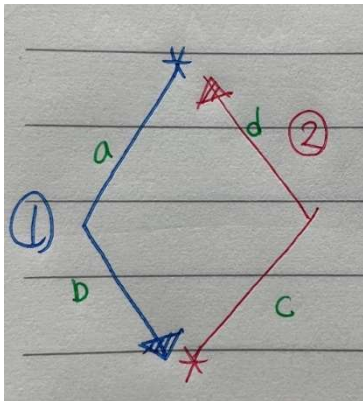
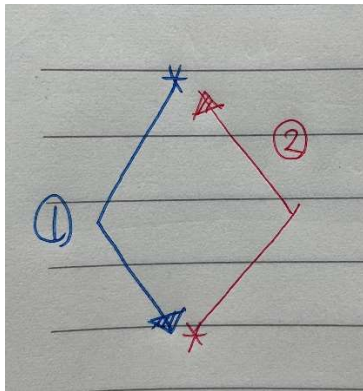
#10번 문제

- 문제해석

1~19사이의 홀수값을 입력받으면 그 숫자를 행 개수로 갖는 다이아몬드 모양을 출력하는 프로그램을 만들어야하는 문제이다.

- 알고리즘

다이아몬드 <> 를 왼쪽과 오른쪽을 나누어 아래와 같은 순서로 출력했다.



이렇게 출력하기 위해서는 먼저 2중 포문으로 문자열안에 공백을 모두 찍는다.

그리고 ①번을 먼저 찍을 것이다. 첫번째 행 중간에 있는 별을 찍고시작한다. 이때 번지수는 $spa[0][row/2]$ 이다. row는 항상 홀수이기 때문에 이렇게 하면 중간에 별이 찍힌다. For문으로 배열을 순회하면서 별을 만났을때 조건에 따라 어떤 위치에 별을 찍을 것인지 정해준다. 만약 ①번 과정에서 $i < row/2$ 일때, 즉 아래그림에서 a 일때는 $spa[i + 1][j - 1] = '*'$ 로 별을 찍어주고 b일 경우엔 $spa[i + 1][j + 1] = '*'$ 로 별을 찍어준다.

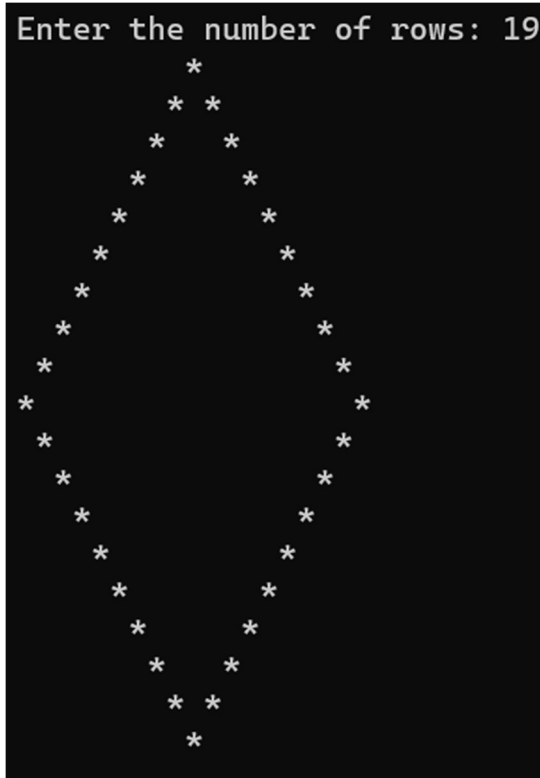
②번을 찍는 방식은 ①번과 비슷하지만 하나 주의해야 할 점이 있다. 똑같이 for 문으로 배열을 돌면서 if 별을 만났을 때, 조건에 따라 별을 찍으라고 시키면 우리가 원하는 별만 찍히는 것이 아니라 행에서 쭉가서 우리가 ①번을 수행하면서 찍고내려왔던 별들과도 만나 다이아몬드 안에도 별이 채워져 버린다.

따라서 ②번을 찍을 때는 if 문 안에서 별을 찍고 break;를 넣어 주어야 한다. 내가 원하는 별만 딱 찍고 나올 수 있도록

- 결과 및 고찰 분석

```
Enter the number of rows: 5
      *
     * *
    *  *
   *  *
  *  *

Enter the number of rows: 3
 *
* *
 *
```



별을 어떻게 찍을지 고민하며 문제의 해결방안을 떠올리는 능력이 소소하게나마 상승했음을 느꼈다. 처음에는 다이아몬드를 윗삼각형 아랫삼각형으로 나눠 찍으려고 했지만 이걸 테두리로 생각해 왼쪽 테두리는 위에서 아래로 아래쪽 테두리는 아래에서 위로 찍는 방식이 떠올랐고 문자열로 찍고 꺼내는걸로 구현이 가능할꺼 같았다.

그렇게 코딩하며 크게 두가지 오류가 났었고 이를 바로잡으며 코딩을 완성했다 첫번째 오류는 2번을 찍기 시작할 때 1번이 종료한 자리에서 다시 시작하려고 했다. 직관적으로 별이 거기서 끝났으니 거기서 끝내야 할것같아서 그렇게 했지만,이렇게 하는 것 아니라 맨아래 맨오른쪽에서 포문 조건을 시작해 주어야 오류가 수정되고 다이아몬드가 제대로 출력된다.

두번째 오류는 2번방향으로 별을 찍을 때 if 문안에 break를 걸어주지 않은 것이다. 위에 알고리즘 설명에서도 언급했듯이, 똑같이 for 문으로 배열을 돌면서 if 별을 만났을 때, 조건에 따라 별을 찍으라고 시키면 우리가 원하는 별만 찍히는 것이 아니라 행에서 쭉가서 우리가 1번을 수행하면서 찍고내려왔던 별들과도 만나 다이아몬드 안에도 별이 이상하게 채워져 버린다. 따라서 2번 방향으로 별을 찍을 때는 꼭 break를 넣어 주어야 한다. 이렇게 안되는 이유를 고뇌하면서 나의 역량이 약간이나마 늘었다고 생각한다.

#11번 문제

- 문제 설명

정수 두 개를 입력받아 연산하는 `multiple`이라는 함수를 만든다. 이 함수는 두번째 숫자가 첫 번째 정수의 배수인지 확인하고, 그렇다면 `true`를 반환, 배수가 아니라면 `false`를 반환하도록 코드를 작성한다.

- 알고리즘

이 코드를 작성하기 위해 `bool`을 이용하였다. `Bool`은 `boolean`으로 참과 거짓을 판단할때 사용하고 0과1의 값만 가지고 있다.

두번째 정수가 첫번째 정수의 배수인지 아닌지를 알아보기 위해서는 두번째수를 첫 번째수로 나눈값의 나머지가 0이면 배수, 아니면 배수가 아닌 것으로 코드를 구현하여 준다. 조건에 해당하면 `1=> true`를 출력하고 해당되지 않으면 `0 => false`를 출력한다.

- 결과 및 고찰 분석

```
Enter the 1st number: 4
Enter the 2nd number: 16
multiple(4,16):true
```

```
Enter the 1st number: 5
Enter the 2nd number: 19
multiple(5,19):false
```

`Bool`을 사용하여 참과 거짓을 출력하는방식을 익혔다. 그리고 `bool`에 대해 알아보던 중 `bool`에서의 0,1과 `int`에서의 0,1은 다르다는걸 깨달았다. 4바이트를 갖는 `int` 형과는 다르게 `bool`은 1바이트를 갖는다고 한다. 분석은 위에 알고리즘 카테고리에 겹쳐 썼다.

#12번 문제

- 문제 해석

제시된 피보나치 수열의 n 번째 피보나치 수를 재귀함수를 사용한 버전과 사용하지 않은 버전을 나누어 연산하고 출력한다. 피보나치 수열은 0,1,1로 시작한다.

- 알고리즘

(a) :재귀를 사용하지 않고 피보나치 수를 구하려면 `for` 문을 사용하여 `a,b,c, ...` 있을때

`c=a+b`를 이용하여 푼다. `For` 문의 i 값이 증가하여 반복하면 한칸씩 옆으로 숫자들을

미뤄서 저장하여 값을 구해나간다. 실질적인 연산은 3항부터 시작하기 때문에 앞에 두값에 대해서는 예외로 if문으로 처리해준다.

(b) : 재귀를 사용하여 피보나치 수를 구하도록 코드를 작성한다.

$\text{fiborec}(n) = \text{fiborec}(n - 1) + \text{fiborec}(n - 2)$ 으로 재귀함수를 설정한다.

- 결과 및 고찰 분석

```
Enter the number: 8
Fibonacci_iter(8): 13
Fibonacci_rec(8): 13
```

```
Enter the number: 9
Fibonacci_iter(9): 21
Fibonacci_rec(9): 21
```

피보나치수를 재귀와 비재귀로 나누어서 풀면서 차이에 대해 고민해 보았다. 먼저 (a)는 함수 하나를 만들어 연산해 주었고, (b)는 함수안에 함수안에 그걸 또 함수안에,,, 이런식으로 연산하도록 되어있다. 그렇다면 어떻게 좋은 연산일까? 각각의 상황에 맞게 다르게 사용하면 될 것이다. 어떤 상황에 어떤 연산법이 좋을지 연구하는것도 컴퓨터공학도가 갖춰야할 마인드라고 생각이 들었다. 피보나치를 사용하면 가지를 따라따라 내려 오고 그걸 모두 연산해 줘야 하기에 다처리하기 더 어려워 보인다. 그러나 아랫값들을 알고 큰값을 계산할 수 있는 상황이라면 피보나치가 좋을 것이다. 물론 이 문제 상황에서서의 성능은 (a)가 좋다고 생각되지만 말이다. 앞으로도 어떤 방식이 어떤 이유에서 적합한 건지 고민해보는 공학도의 마인드를 가져야겠다.

#13번 문제

- 문제 해설

표준 라이브러리 함수를 사용하지 않고, 대문자를 소문자로 , 소문자를 대문자로 변환하는 프로그램을 작성하는 문제이다. 입력 문자열은 최대 100자로 구성될 수 있다.

- 알고리즘

대소문자 변환은 아스키 코드를 사용하여 구한다. 아스키 코드에서 대문자와 소문자는 32만큼 차이가 난다. $A=65$, $a=97$ 이렇게 각 알파벳에 32만큼 더하거나 빼주면 대소문자 변환이 가능하다. ($A \sim Z=65 \sim 90$, $a \sim z=97 \sim 122$) 이다. 따라서 만약 입력받은 값이 문자 'A' ~ 'Z' 의 정수형값의 사이에 있을때는 값에다가 32만큼 더해주고 'a'~'z' 정수형값의 가이 값일 때는 32만큼 빼준다.

터득할 수 있었다. 그리고 분석은 위에 알고리즘 카테고리에 겹쳐썼다.

#15번 문제

- 문제 해설

구게임을 구현하는 것이다. 이 게임은 0~9 사이에 4개의 난수를 겹치지 않게 생성해 줘야한다. 이 숫자는 표시 되지 않고 4개의 숫자를 입력해가며 맞추는 게임이다. 난수와 추측숫자 사이에 동일한 위치에 동일한 숫자가 있다면 hit, 잘못된 위치에 숫자가 있다면 'blow' 를 표시한다. 5번의 기회 내에서 올바른 숫자를 얻으면 이 게임을 이겼다고 win을 출력한다. 정답은 실행할때마다 달라야 한다.

- 알고리즘

이 알고리즘 을 구현하기 위해 첫 번째로는 난수를 받아야 한다. 난수는 srand (time)으로 난수생성을 해준다. 배열에 포문을 이용해 하나씩 값을 저장하는데 `secret[i] = rand() % 10;`를 이용하여 해준다.

그러나 이 난수값은 겹치면 안된다. 그러려면 포문을 사용해 각 자릿수자를 검사하고 다음과 같이 같다면 `if (secret[i] == secret[j])` , i를 취해줘 다시 돌아가서 난수로 숫자를 배정받도록 한다.

그리고 숫자를 어떻게 찢어 배열에 하나씩 저장할지는 나누기와 나머지 연산을 사용했다.

0번지 → `input / 1000` 1번지 → `(input % 1000) / 100`

2번지 → `(input % 100) / 10` 3번지 → `input % 10`

(0번지는 천의자리~3번지는 일의자리 이다.)

그리고 hit blow 판단은 먼저 hit일때는 `secret[j] == guess[j]` 일때를 hit 로 카운트 해주었다. 그리고 blow 는 for 문과 if 문으로 `secret[j] == guess[h] && j != h` 이럴경우 blow를 하나씩 증가시키는 방법으로 카운트 해주었다. 뒤는 blow에 hit 가 겹칠 경우를 빼주기위함이다. 마지막 win/lose 는 if 문을 사용해 hit 가 4일 경우 win을 출력하도록 해주었다.

- 결과 및 고찰 분석

```
Guess: 1234
Hit: 2 Blow: 0
-----

Guess: 5678
Hit: 0 Blow: 1
-----

Guess: 9012
Hit: 1 Blow: 0
-----

Guess: 3467
Hit: 0 Blow: 3
-----

Guess: 1673
Hit: 0 Blow: 2
-----

Lose

the correct answer: 9734
```

분석은 위에 알고리즘 카테고리에 겹쳐썼다. 이 문제는 코딩설계가 익숙하지 않았던 나에게 정말 어려웠던 문제이다. 특히 hit blow를 판단해줄 때 머리로, 손으로 단계별로 차근차근 썼을 때는 될것만 같던게 코드로 구현하니 오류가 계속 나는 모습에 정말 답답했다. 그러나 blow 조건에 hit 가 겹칠때를 빼줘야 한다는걸 깨달았을 때 정말 기뻐다. 그리고 겹치치 않는 난수를 출력하기 위한 코드작성을 생각해 냈을때도 정말 기뻐다. 그런식으로 논리적 사고력을 늘려가기 위한 노력노 하는 것이 앞으로 내가 가져야할 태도인 것 같다.